

Name: Richmond Kobby Nyarko

Deploying a Sample Application with Kubernetes

The Objective is to deploy a sample application on a Kubernetes cluster, learn how to manage deployments, and gain hands-on experience with scaling and service exposure.



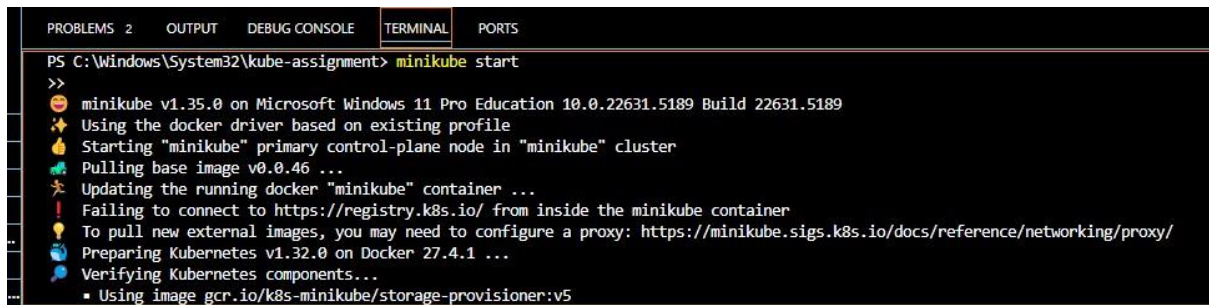
```
1  const express = require('express');const app = express();const port = 3000;
2
3  app.get('/', (req, res) => {
4    | res.send('Hello from Kubernetes Node.js App!');
5  });
6
7  app.get('/status', (req, res) => {
8    | res.json({ status: 'OK', timestamp: new Date() });
9  });
10
11 app.listen(port, () => {
12   | console.log('App running on http://localhost:${port}');
13 });
```

☑ Task 2: Environmental Setup

1. Install Minikube: <https://minikube.sigs.k8s.io/docs/start/>
2. Start Minikube:

Successfully installed

Minikube start command run in order to start minikube below



```
PS C:\Windows\System32\kube-assignment> minikube start
>>
minikube v1.35.0 on Microsoft Windows 11 Pro Education 10.0.22631.5189 Build 22631.5189
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.46 ...
Updating the running docker "minikube" container ...
! Failing to connect to https://registry.k8s.io/ from inside the minikube container
! To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
Preparing Kubernetes v1.32.0 on Docker 27.4.1 ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
```



```
PS C:\Windows\System32\kube-assignment> kubectl version --client
>> kubectl get nodes
>>
Client Version: v1.32.2
Kustomize Version: v5.5.0
NAME          STATUS    ROLES          AGE    VERSION
minikube      Ready     control-plane  10h    v1.32.0
PS C:\Windows\System32\kube-assignment>
```

Kubectl verified successfully

- ☒ **Task 3: Creating Deployment Manifest deployment.yml file that defines a**
- Kubernetes Deployment for the chosen application

```
! deployment.yml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    | name: nodejs-deployment
5  spec:
6    replicas: 2
7    selector:
8      | matchLabels:
9        | app: nodejs-app
10   template:
11     metadata:
12       labels:
13         | app: nodejs-app
14     spec:
15       containers:
16         - name: nodejs-container
17           image: your-dockerhub-username/nodejs-k8s-app:latest
18           ports:
19             - containerPort: 3000
```

- Task 4: Exposing the Application•
- A Service YAML file to expose the deployment

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    | name: nodejs-service
5  spec:
6    type: NodePort
7    selector:
8      | app: nodejs-app
9    ports:
10     - protocol: TCP
11       port: 80
12       targetPort: 3000
13       nodePort: 30036 # Optional: can be auto-assigned
14
```

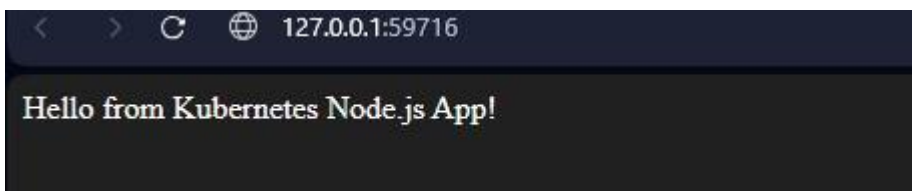
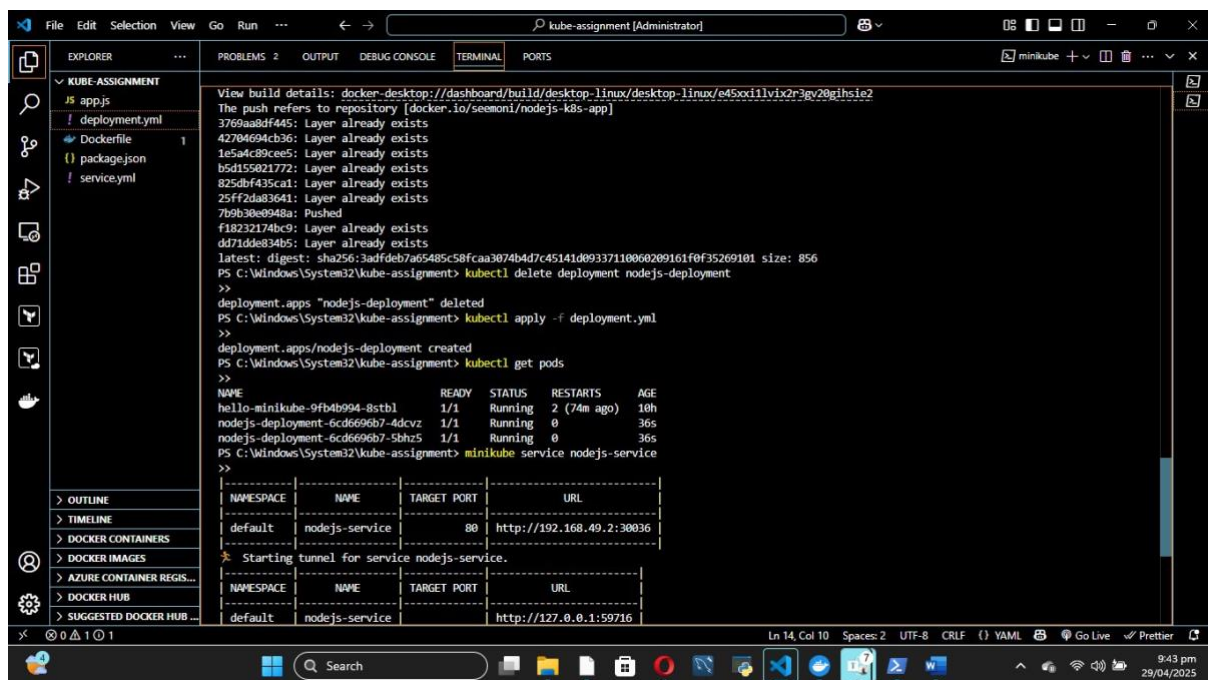
- ☒ **Task 5: Deploying and Testing**

```
PS C:\Windows\System32\kube-assignment> kubectl apply -f deployment.yml
>> kubectl apply -f service.yml
>>
deployment.apps/nodejs-deployment created
service/nodejs-service created
PS C:\Windows\System32\kube-assignment> |
```

Deployment and service files Successfully created

```
PS C:\Windows\System32\kube-assignment> kubectl apply -f deployment.yml
>> kubectl apply -f service.yml
>>
deployment.apps/nodejs-deployment created
service/nodejs-service created
PS C:\Windows\System32\kube-assignment> kubectl get pods
>>
NAME                                READY   STATUS              RESTARTS   AGE
hello-minikube-9fb4b994-8stbl       1/1     Running             2 (36m ago) 9h
nodejs-deployment-5689bb757c-st7zw  0/1     ImagePullBackOff    0           115s
nodejs-deployment-5689bb757c-xzz7g  0/1     ImagePullBackOff    0           115s
```

Pods checked successfully



Scale up successfully

```
PS C:\Windows\System32\kube-assignment> kubectl scale deployment/nodejs-deployment --replicas=5
>>
deployment.apps/nodejs-deployment scaled
PS C:\Windows\System32\kube-assignment> |
```

Effects of Scaling the Deployment

1. Increased Availability and Load Distribution:

- With 5 replicas, the application will be able to handle more traffic since the load will be distributed across 5 separate pods. ○ If you are running a web service, this can help ensure the service remains responsive under increased demand.

2. Improved Fault Tolerance:

- Having multiple replicas ensures that if one pod goes down, the remaining pods can continue to handle the traffic. This reduces the impact of a pod failure.

3. Resource Usage:

- Each replica consumes CPU and memory resources. With 5 replicas, the Kubernetes cluster will need sufficient resources (like CPU, RAM, etc.) to handle the increased load from the additional pods.

4. Network Overhead:

- Scaling up may introduce some network overhead as more pods need to communicate with each other and with the services they depend on.
- Kubernetes may also increase the number of service endpoints, potentially increasing the complexity of internal traffic routing.

Performing a Rolling Update

A rolling update allows you to change the container image of your pods without downtime. This is helpful if you need to deploy a new version of your application without interrupting its availability

```
PS C:\Windows\System32\kube-assignment> kubectl set image deployment/nodejs-deployment nodejs-container=node:latest
>>
deployment.apps/nodejs-deployment image updated
PS C:\Windows\System32\kube-assignment> |
```